GUIDE D'IMPLÉMENTATION · PROJET D'ÉQUIPE

Meal Planner & Food Tracker

Suivi des repas, menus et inventaire du frigo

Spring Boot · Angular · PostgreSQL

Clarisse
Personne 1 : inventaire, alertes et authentification

Leticia
Personne 2 : recettes, unités et infrastructure

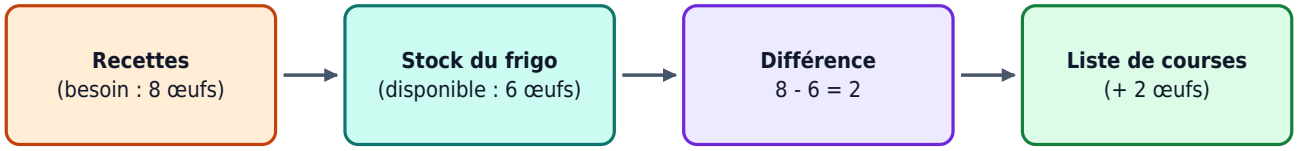
Georgette
Personne 3 : planning, liste de courses et qualité

Version 1.0 · Octobre 2026 · Document de référence pour l'équipe

1. Le projet en bref

Une application web qui sait **ce que vous avez dans le frigo**, **ce que vous voulez cuisiner cette semaine** et qui calcule seule **ce qu'il faut acheter**. Elle prévient aussi quand un aliment va bientôt périmer.

Le scénario de la démo : les œufs



Les **recettes** viennent de **Leticia**, le **stock** de **Clarisse**, le **calcul et la liste** de **Georgette**. Les trois parties doivent s'emboîter parfaitement.

Technologie	Usage
Spring Boot + Hibernate/JPA	API REST et logique métier
PostgreSQL	Base de données (Docker déjà configuré : rien à faire de ce côté)
Angular + Angular CDK	Interface et glisser-déposer du planning

2. Répartition des rôles

On découpe par **domaine fonctionnel** : chacune développe le backend **et** le frontend de son domaine. Personne n'attend l'autre et chacune pratique Spring Boot et Angular.

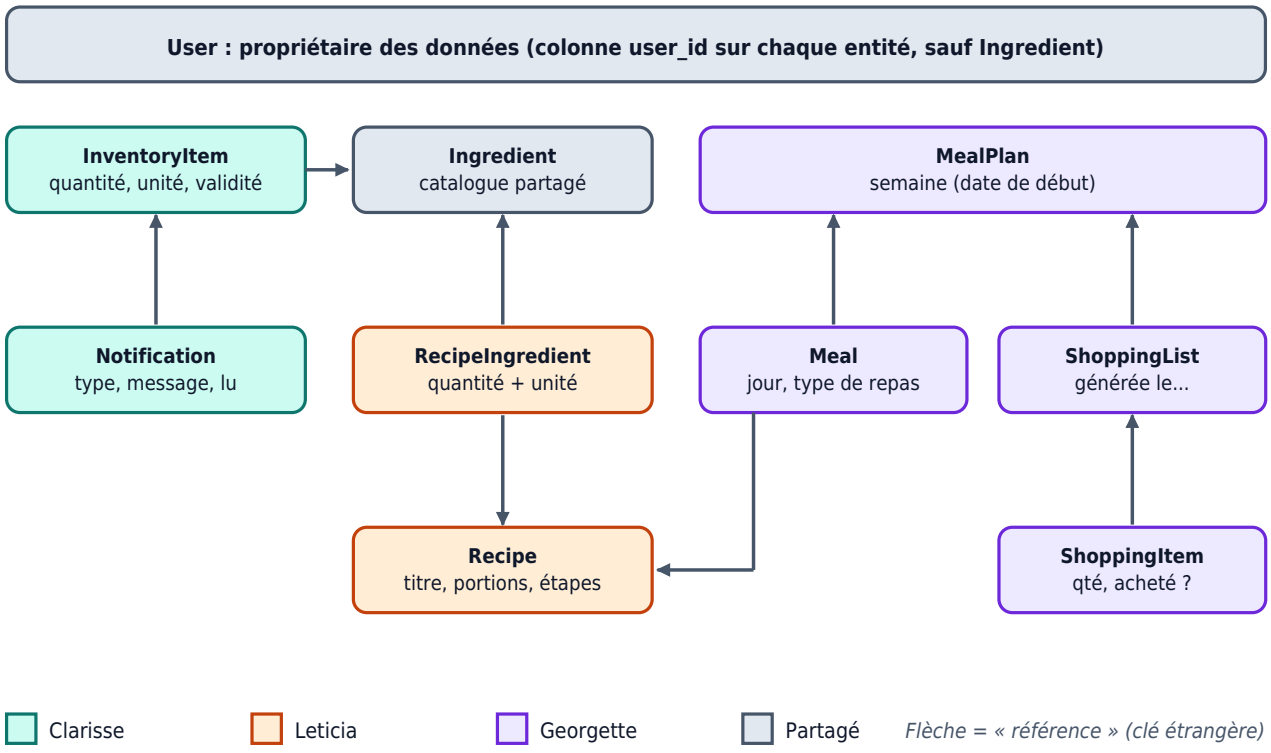
	Domaine	Tâche transversale	À livrer en premier
Clarisse Personne 1	Inventaire et alertes de péremption	Authentification JWT et utilisateurs	Connexion, puis liste de l'inventaire
Leticia Personne 2	Recettes, ingrédients, unités	CI, README, documentation Swagger	Conversion d'unités, puis liste des recettes
Georgette Personne 3	Planning, ingrédients manquants, liste de courses	Tests d'intégration et démo finale	Calendrier, puis calcul des manquants

Relecture croisée des pull requests

Clarisse est relue par **Georgette**, Leticia par **Clarisse**, Georgette par **Leticia**. Personne ne valide son propre travail.

3. Modèle de données

Les couleurs indiquent **qui est responsable** de chaque table (migration, entité et tests). Le schéma se dessine et se valide à trois avant de se séparer.



À retenir

Les quantités sont en **BigDecimal** (jamais en nombre à virgule flottante), les relations en **LAZY**, le schéma évolue uniquement par **migrations Flyway**, et chaque requête filtre sur l'**utilisateur connecté**.

4. Clarisse : inventaire, alertes et authentification

Clarisse · Personne 1

Ce que l'utilisateur possède chez lui

Mission : permettre de se connecter, d'enregistrer ce qu'il y a dans le frigo et les placards, et de prévenir avant que la nourriture ne périme.

Backend	Frontend
• Utilisateur, inscription, connexion (JWT, mots de passe hachés) • Inventaire : CRUD complet • Filtres par lieu et par date de péremption • Tâche quotidienne qui détecte les aliments à expirer • Notifications (liste, marquer comme lue)	• Pages de connexion et d'inscription • Protection des routes et envoi automatique du jeton • Page d'inventaire : liste, filtres, formulaire • Badges d'urgence • Cloche de notifications

Étapes dans l'ordre

1. **Authentification en premier** (semaine 1) : toute l'équipe en dépend.
2. **Inventaire** : entité, migration, API CRUD, validation des données, accès limité aux données de l'utilisateur connecté.
3. **Alertes** : une tâche planifiée chaque matin crée une notification pour les aliments qui expirent bientôt (délai configurable), sans doublon.
4. **Frontend** : d'abord connexion et protection des routes, puis page d'inventaire, puis cloche de notifications.
5. **Fournir à Georgette** le total disponible par ingrédient (dans une unité de base), via une interface convenue ensemble.

Badge	Condition
Rouge : expiré	date de péremption passée
Orange : bientôt périmé	expire dans les 3 prochains jours (valeur configurable)
Vert : OK	au-delà, ou sans date

C'est fini quand...

- Un utilisateur ne voit que son propre inventaire (testé avec deux comptes).
- Ajouter 6 œufs qui expirent dans 2 jours fait apparaître une alerte.
- Les erreurs de saisie (quantité négative, etc.) renvoient un message clair.

Pièges à éviter

Renvoyer le mot de passe haché dans une réponse ; oublier le filtre par utilisateur ; mettre la logique des alertes dans la classe planifiée plutôt que dans un service ; écrire la clé secrète JWT dans le code.

5. Leticia : recettes, unités et documentation

Leticia · Personne 2

Le catalogue de recettes et le système d'unités

Mission : gérer les recettes avec leurs ingrédients, donner à toute l'équipe un système d'unités fiable, et assurer la CI, le README et la documentation de l'API.

Backend	Frontend
• Ingrédients (catalogue partagé) et recettes • CRUD des recettes avec leurs ingrédients • Recherche, filtres et pagination • **Conversion d'unités** (g, kg, mL, L, pièce...) • Documentation Swagger	• Liste de recettes et page de détail • Formulaire dynamique (ajouter / retirer des ingrédients) • Recherche et auto-complétion des ingrédients

Étapes dans l'ordre

1. **Unités d'abord** (semaine 1) : à publier tout de suite, Clarisse et Georgette en dépendent.
2. **Catalogue d'ingrédients** avec données de test (œufs, poulet, pâtes, tomates).
3. **Recettes** : entités, CRUD, recherche paginée, chargement efficace des ingrédients.
4. **Frontend** : liste, détail, puis formulaire dynamique.
5. **CI, README et Swagger** : chaque pull request lance build et tests automatiquement.

Pourquoi les unités sont cruciales

Une recette demande **0,5 kg** de poulet, le stock contient **500 g** : il ne manque rien. Sans conversion, l'application comparerait « 0,5 » et « 500 » et se tromperait. Chaque unité appartient à une **famille** (masse, volume, comptage) et se convertit vers une **unité de base**. On ne convertit jamais d'une famille à une autre.

Famille	Unités	Unité de base
Masse	g, kg	g (1 kg = 1000 g)
Volume	mL, L, cuillère à café, cuillère à soupe	mL (1 L = 1000 mL)
Comptage	pièce	pièce (ex. 6 œufs)

C'est fini quand...

- On crée, modifie et supprime une recette « Omelette » (4 œufs, 20 g de beurre).
- La recherche « tom » retrouve « Salade de tomates ».
- Les conversions sont testées à fond (1 kg = 1000 g ; g et mL incompatibles).

Pièges à éviter
Renvoyer directement les entités au frontend (boucles infinies) ; charger les ingrédients recette par recette (lenteur) ; modifier la conversion d'unités sans prévenir les deux autres.

6. Georgette : planning, liste de courses et qualité

Georgette · Personne 3

Le cœur de la logique métier et la démonstration

Mission : planifier les repas de la semaine par glisser-déposer, détecter ce qui manque dans le stock et produire automatiquement la liste de courses.

Backend	Frontend
• Plan de la semaine et repas • Calcul des ingrédients manquants • Génération de la liste de courses • Déplacement d'un repas d'un jour à l'autre	• Calendrier semaine (7 jours × 3 repas) • Glisser-déposer avec Angular CDK • Résumé des manquants • Liste de courses avec cases « acheté »

Étapes dans l'ordre

1. **Avancer sans attendre** (semaine 1) : définir avec Clarisse et Leticia les « contrats » (interfaces) du stock et des recettes, et travailler d'abord avec des données factices.
2. **Entités du planning** : un plan par utilisateur et par semaine, des repas (jour, type, recette, portions).
3. **Calcul des manquants** : algorithme en 4 temps ci-dessous, avec tests unitaires.
4. **Liste de courses** : régénérable sans doublons, en conservant les éléments déjà cochés.
5. **Frontend** : calendrier fixe, puis glisser-déposer, puis branchement à l'API, puis liste de courses.
6. **Qualité** : test d'intégration du scénario des œufs, puis script de démo.

L'algorithme en 4 temps

Exemple : omelette (4 œufs) + gâteau (4 œufs) dans la semaine ; stock de 6 œufs.

Temps	Action	Exemple
1. Besoins	Additionner les ingrédients de toutes les recettes du plan (ajustés aux portions), en unité de base	8 œufs
2. Stock	Additionner l'inventaire par ingrédient, en unité de base, sans les aliments périmés	6 œufs
3. Différence	Manquant = besoin - stock, si positif	8 - 6 = 2
4. Liste	Un élément de courses par manquant, converti en unité lisible	2 œufs

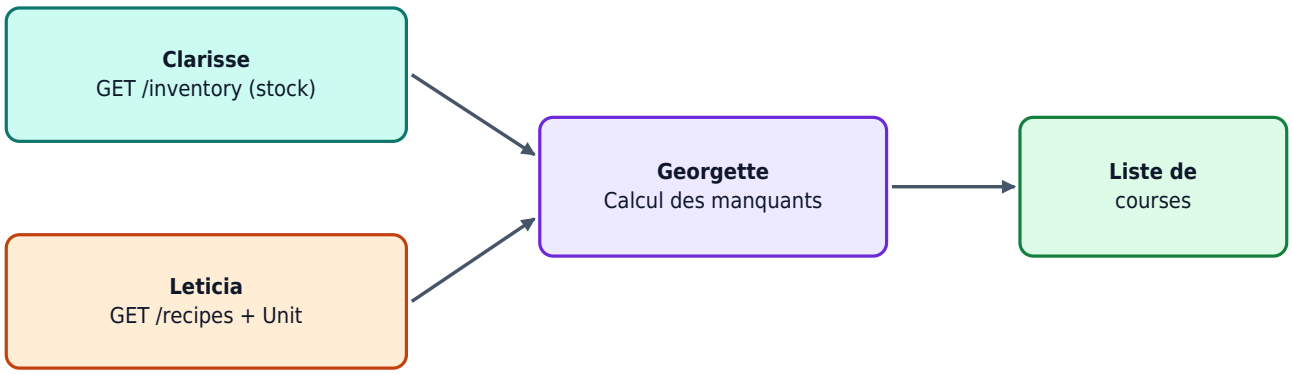
C'est fini quand...

- Glisser un repas du mardi au jeudi met à jour la base.
- Le scénario des œufs (8 - 6 = 2) est couvert par un test automatisé.
- Si le stock suffit, la liste est vide et l'application l'indique.

Pièges à éviter

Comparer des quantités sans les convertir ; soustraire le stock avant d'avoir additionné tous les repas qui utilisent le même ingrédient ; attendre les autres au lieu de travailler avec des données factices.

7. Intégration et contrat d'API



Georgette dépend des deux autres : elles livrent d'abord les GET + les données de test (seed).

Ordre de livraison pour ne bloquer personne

Ordre	Livraison	Qui	Débloque
1	Authentification	Clarisse	Toutes les pages protégées
2	Conversion d'unités	Leticia	Calculs de Clarisse et Georgette
3	Lecture de l'inventaire et des recettes + données de test	Clarisse, Leticia	Georgette quitte les données factices
4	Planning + calcul des manquants	Georgette	Liste de courses et démo

Les endpoints (préfixe /api/v1)

Responsable	Endpoints principaux
Clarisse	POST /auth/register, POST /auth/login ; GET, POST, PUT, DELETE /inventory ; GET /notifications ; PATCH /notifications/{id}/read
Leticia	GET, POST /ingredients ; GET, POST, PUT, DELETE /recipes (recherche et pagination)
Georgette	GET /meal-plans/{semaine} ; POST /meal-plans/{id}/meals ; PATCH /meals/{id}/move ; GET /meal-plans/{id}/missing-ingredients ; POST /meal-plans/{id}/shopping-list ; PATCH /shopping-items/{id}

Toute modification d'un endpoint ou d'un modèle partagé est annoncée aux deux autres **avant** d'être codée.

Données de test partagées

Un seul jeu de données pour tout le monde : un utilisateur de test, un stock avec **6 œufs et 500 g de poulet**, et deux recettes qui demandent **8 œufs** au total. Chacune voit ainsi le même scénario.

8. Bonnes pratiques

Domaine	Règles à respecter
Git	Branches **main** (stable) ← **develop** ← **feature/*** ; jamais de push direct. Commits clairs (feat, fix, test). Petites pull requests relues par une autre personne, avec la CI verte. Mise à jour depuis develop chaque matin.
Backend	Couches Controller → Service → Repository ; logique métier uniquement dans les services. Objets de transfert (DTO) en entrée et sortie, jamais d'entités. Validation des données, gestion d'erreurs unique, relations LAZY, migrations Flyway, configuration par variables d'environnement.
Frontend	Un dossier par fonctionnalité, routes chargées à la demande, aucun appel HTTP dans les composants (passer par un service), types TypeScript pour chaque DTO, formulaires réactifs, un intercepteur pour le jeton et les erreurs.
Tests	Tests unitaires sur les services (conversions, calcul des manquants), test d'intégration du scénario des œufs, objectif d'environ 70 % de couverture sur les services.
Sécurité	Mots de passe hachés, secrets hors du code, jeton avec expiration, vérification que chaque ressource appartient à l'utilisateur, CORS limité.
Qualité	Noms explicites, méthodes courtes, pas de duplication (le partagé va dans le dossier commun), commentaires pour le « pourquoi ».

9. Planning sur 4 semaines

	Clarisse	Leticia	Georgette	Jalon d'équipe
Semaine 1	Authentification ; inventaire (backend)	Unités ; ingrédients + données de test ; CI	Entités du planning ; données factices ; calendrier fixe	On peut se connecter
Semaine 2	Page d'inventaire ; protection des routes	Recettes (backend) ; liste, détail, formulaire	Endpoints du plan ; ajout de repas	Inventaire et recettes disponibles : fin des données factices
Semaine 3	Alertes et notifications ; cloche	Recherche, filtres, Swagger	Glisser-déposer ; manquants ; liste de courses	**Scénario des œufs** de bout en bout
Semaine 4	Tests ; e-mail d'alerte (optionnel)	Tests ; README final	Tests d'intégration ; script de démo	Démo répétée 2 fois sans accroc

Si on prend du retard
On coupe d'abord l'optionnel (e-mail, filtres secondaires), jamais le scénario principal. Priorité : **Inventaire → Recettes → Calcul des manquants → Liste de courses**.

10. Terminé et démo finale

Une tâche est terminée quand : la pull request est relue, les tests passent, les cas d'erreur sont gérés, l'endpoint est documenté dans Swagger et la tâche est passée en « Terminé » sur le tableau Kanban.

Script de la démo (5 à 7 minutes)

1. **Clarisse** : connexion ; inventaire avec 6 œufs et 500 g de poulet ; badge orange sur un aliment bientôt périmé.
2. **Leticia** : création rapide d'une recette avec le formulaire dynamique ; recherche.
3. **Georgette** : planning de la semaine ; glisser-déposer d'un repas ; génération de la liste : **2 œufs apparaissent**.
4. **Clarisse** : la cloche montre l'alerte de péremption. **Toutes** : conclusion et questions.